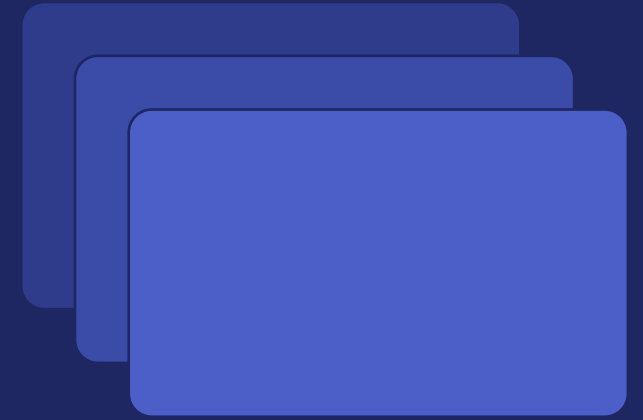




Container and Image Security in Docker

Session Overview; What We will Build, Break, and Harden

What We'll Cover

1

Image Security

Hygiene, digest pinning, vulnerability scanning

2

Docker Hardened Images

Minimal, low-CVE drop-in replacements

3

Secrets Management

Keeping credentials out of image layers

4

Multi-Stage Builds

Separating build tools from runtime

5

Dockerfile Best Practices

Non-root, read-only, pinned versions

6

Runtime & Provenance

Capabilities, SBOMs, no privileged mode

پک مرور کوچول

پک مرور کوچول



پیک مرور کوچول

Dockerfile

```
docker build -t fozouni/compose-demo:v1 .
```

Docker Image

Docker compose

```
docker compose up -d
```

Web service

Redis

PostgreSQL

**ممکن است نیاز به مرور چندین
باره‌ی مطالب این فایل پیدا کنید.**

موارد مطرح شده در ادامه تنها جهت ارائه‌ی یک کلیت از کارهای پیش رو هستند.

1- Image Security

Periodic Re-Scans



Vulnerability Scanning

Base Image Hygiene



Rebuild Regularly



1- Image Security

Start clean, stay patched, catch problems before they ship

- **Base Image Hygiene** Minimal official images, pinned by digest; tags are mutable, digests aren't.
- **Rebuild Regularly** A "frozen" image silently rots as new CVEs surface upstream. Rebuild to repull patches.
- **Vulnerability Scanning** Trivy & Docker Scout scan OS packages + language deps; fail CI on Critical/High findings.
- **Periodic Re-Scans** New CVEs are disclosed after the fact; re-scan images already sitting in the registry.

2- Docker Hardened Images (DHI)



2- Docker Hardened Images (DHI)

Minimal, hardened, drop-in replacements

12x

smaller image size

6x

fewer packages

0

critical vulnerabilities

Two variants for every language image:

dev --- build tools, shell, root user (build stage)

runtime --- stripped down, production-ready

چرا این دو تا جدا شدن؟

برای اینکه در مرحله‌ی ساخت (build) به ابزار زیاد نیاز داریم، اما در محیط اجرای نهایی (runtime) فقط به حداقل فایل‌ها نیاز داریم تا حمله‌پذیری کمتر، حجم کمتر و سرعت بالاتر برود. این یعنی اصل حداقل دسترسی (Principle of Least Privilege) در امنیت.

3- Secrets Management



AVOID

Secrets baked into the image
(visible forever)

Dockerfile

```
1 FROM ubuntu:22.04
2 ARG API_KEY=sk_live_12345
3 ENV DB_PASSWORD=supersecret
4 RUN echo $API_KEY > /app/key.txt
```



Secrets baked into layers



DOCKER IMAGE HISTORY	
layer 4	RUN echo \$API_KEY > /app/key.txt
layer 3	ENV DB_PASSWORD=supersecret
layer 2	ARG API_KEY=sk_live_12345
layer 1	FROM ubuntu:22.04



Secrets visible forever



Anyone with access to the image history can extract secrets



USE INSTEAD

Secrets passed securely
(not stored in image)

Dockerfile

```
1 FROM ubuntu:22.04
2 RUN --mount=type=secret,id=API_KEY \
3     some-command-that-uses-secret
4 CMD ["app"]
```



Secret used at build time only



CLEAN IMAGE HISTORY

layer 3	CMD ["app"]
layer 2	RUN some-command
layer 1	FROM ubuntu:22.04



No secrets in history

PASS SECRETS SECURELY AT RUNTIME



--env-file
e.g. docker run --env-file .env

OR



-e
e.g. docker run -e API_KEY=...

OR



Secrets Manager
e.g. Docker Swarm / Kubernetes Secrets

3- Secrets Management

Once a secret is in a layer, it's in the image history forever

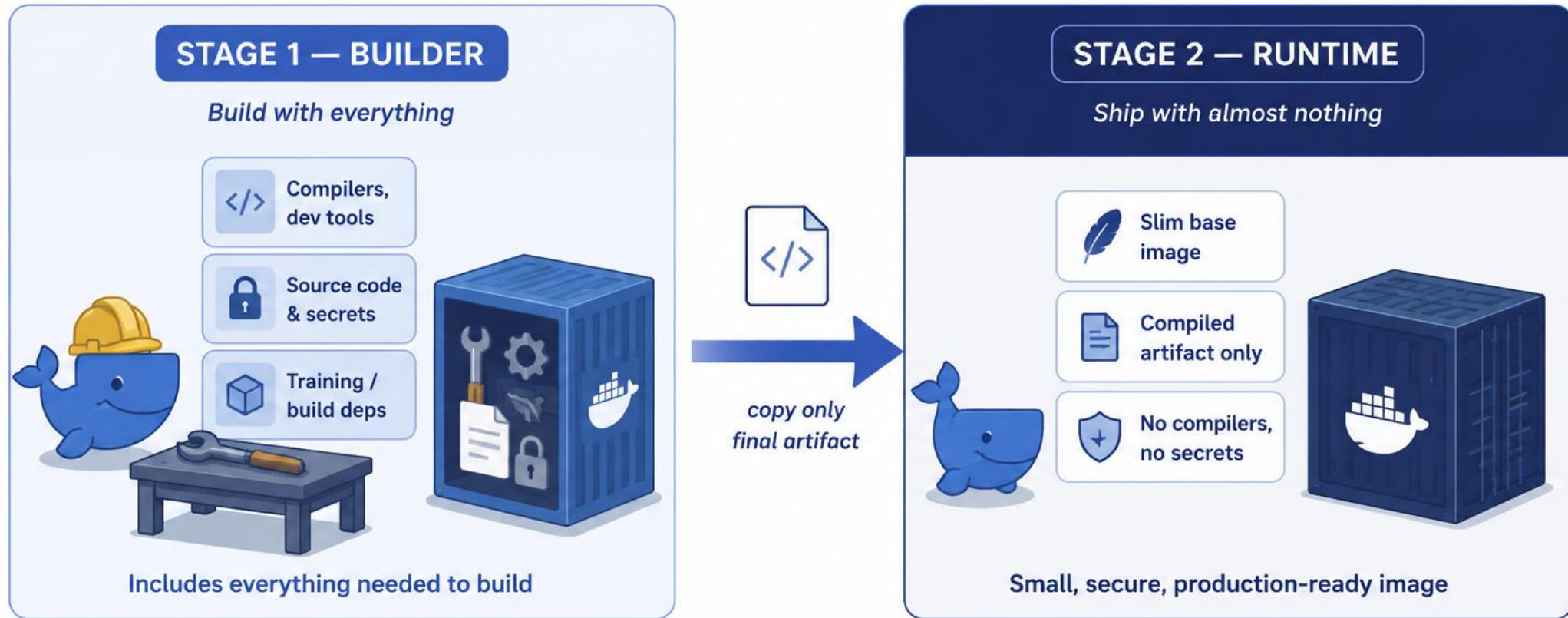
Avoid

- COPY .env / keys / tokens into an image
- ARG / ENV for credentials --- visible via docker history
- Assuming RUN rm secret.txt removes it (it doesn't)

Use Instead

- BuildKit --mount=type=secret at build time
- Pass tokens via --env-file or -e at runtime
- .dockerignore for .env, keys, and datasets

4- Multi-Stage Builds



4- Multi-Stage Builds

Build with everything, ship with almost nothing

Stage 1 --- Builder

Compilers, dev tools
Source code & secrets
Training / build deps



*copy only
final artifact*

Stage 2 --- Runtime

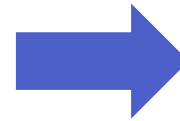
Slim base image
Compiled artifact only
No compilers, no secrets

4- Multi-Stage Builds

Build with everything, ship with almost nothing

Stage 1 --- Builder

Compilers, dev tools
Source code & secrets
Training / build deps



*copy only
final artifact*

Stage 2 --- Runtime

Slim base image
Compiled artifact only
No compilers, no secrets



VERIFY: docker history should show none of Stage 1's build tools in the final image.



```
docker history <final-image>
... gcc 350MB
... git 120MB
... build-tools 200MB
... app 25MB
```



```
docker history <final-image>
... base image 15MB
... app 25MB
```



Clean, minimal,
and secure!

5- Dockerfile Best Practices



1 Non-Root User

USER appuser; never leave containers running as root.



2 Read-Only Filesystem

--read-only at runtime, with explicit writable volumes for /tmp, caches.



3 .dockerignore

Exclude .git, .env, datasets, notebooks — keep secrets out of the build context.



4 No Remote ADD

Prefer COPY + checksum-verified downloads over ADD with remote URLs.



5 Pin Versions

pip install torch==2.3.1; avoid floating versions and dependency confusion.



6 Verify Checksums

Validate signatures/hashes of downloaded models or datasets at build time.

5- Dockerfile Best Practices

1 Non-Root User

USER appuser; never leave containers running as root.

2 Read-Only Filesystem

--read-only at runtime, with explicit writable volumes for /tmp, caches.

3 .dockerignore

Exclude .git, .env, datasets, notebooks — keep secrets out of the build context.

4 No Remote ADD

Prefer COPY + checksum-verified downloads over ADD with remote URLs.

5 Pin Versions

pip install torch==2.3.1; avoid floating versions and dependency confusion.

6 Verify Checksums

Validate signatures/hashes of downloaded models or datasets at build time.

6- Runtime Security & Provenance

Runtime Hardening



--cap-drop=ALL

Drop all Linux capabilities,
add back only what's needed



--security-opt=no-new-privileges

Blocks setuid escalation (sudo, su)



Never --privileged

Disables container isolation entirely

Supply-Chain Provenance



SBOM (Software Bill of Materials)

Tracks every dependency shipped
in the image



docker scout / trivy --format cyclonedx

Generate & store as a build artifact
for audits



Docker socket

Never mount it into a container –
it's root on the host

6- Runtime Security & Provenance

Runtime Hardening

```
--cap-drop=ALL  
Drop all Linux capabilities, add back only what's  
needed  
  
--security-opt=no-new-privileges  
Blocks setuid escalation (sudo, su)  
  
Never --privileged  
Disables container isolation entirely
```

Supply-Chain Provenance

```
SBOM (Software Bill of Materials)  
Tracks every dependency shipped in the image  
  
docker scout / trivy --format cyclonedx  
Generate & store as a build artifact for audits  
  
Docker socket  
Never mount it into a container – it's root on the host
```



Production-safe baseline:

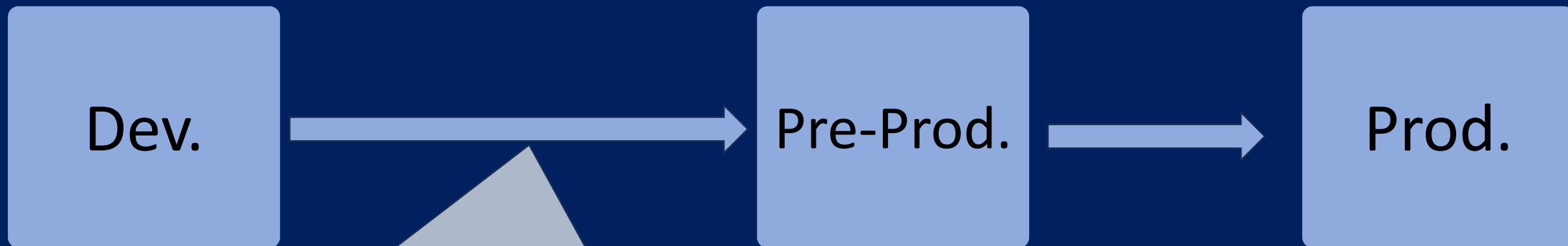
```
docker run --cap-drop=ALL --security-opt=no-new-privileges -u appuser myapp
```

اس بی او ام به شما می گوید «چه چیزی، با چه نسخه ای، در کجاست» تا وقتی خطری پیش می آید، بیدارنگ بدانید آیا تحت تأثیر قرار گرفته اید یا نه و چطور رفعش کنید. بدون این مولفه انگار یک جعبه ی سیاه داریم که نمی دانیم داخلش چیست.

Whole picture of this story

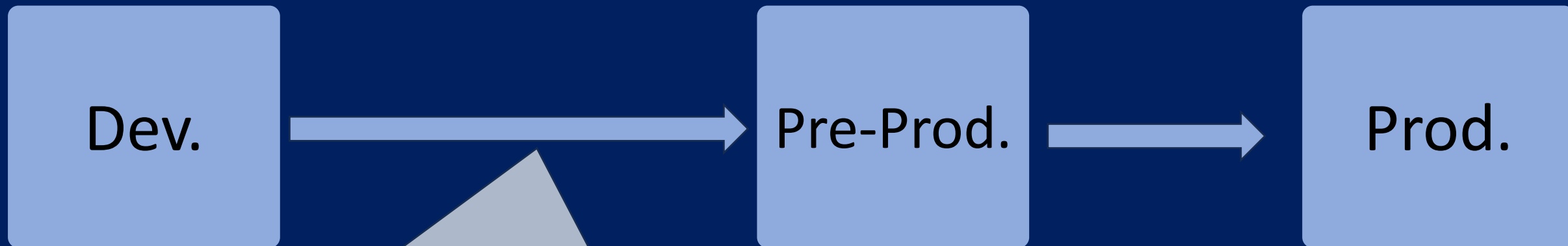


Whole picture of this story



در این مرحله هست که باید سعی کنیم همه چیز رو اتومیت یا خودکار کنیم. از جمله:

- بررسی خودکار ایمیجها
- وجود رمز یا سیکرت خاص داخل آنها یا سورس کد قبل از ارسال به گیت‌هاب
- چک کردن مسائل امنیتی اپ پایتونی
- اسکن فایل و دایرکتوری‌ها
- ...



در این مرحله هست که باید سعی کنیم همه چیز رو اتومیت یا خودکار کنیم. از جمله:

- بررسی خودکار ایمیج‌ها
- وجود رمز یا سیکرت خاص داخل آنها یا سورس کد قبل از ارسال به گیت‌هاب
- چک کردن مسائل امنیتی اپ پایتونی
- اسکن فایل و دایرکتوری‌ها
- ...

این خودکار سازی چک نمودن ابزارهای موجود در سرویس از DevSecOps میاد. ما در MLSecOps دقیقن همین کارها رو باید برای مدل‌ها و دیتای موجود در سیستم خودمون هم تنظیم کنیم که انجام شوند. وگرنه به جای سیستم خواهد لنگید.



تمرکز کنید روی کاری که انجام می شود.
نه ابزار خاص انجام آن کار.

کلیت کار هیچگاه تغییر نخواهد کرد. اما ابزارها، بله.

Now let's goooo for practical stuffs in the “.md” file 🚀



Container and Image Security in Docker

MLSecOps in Action-Mohammad Fozouni